DevMentor AI - By Dhruv Shukla

13 Aug 2026

Daily Learning Report

TODAY'S LEARNING

Interfaces

Day 7 | Intermediate | 10 min read

Generated by DevMentor AI by Dhruv Shukla

Today's Goal

After today, Dhruv will be able to design loosely coupled, testable C# applications using interfaces as contracts. He will understand internal interface dispatch in the .NET runtime, configure dependency injection using abstractions in ASP.NET Core, and avoid common design pitfalls like interface bloat and tight coupling.

Core Concept

An interface is a contract in C# that defines what set of capabilities a type provides without committing to how those capabilities are implemented. It solves the problem of tight coupling—allowing callers to depend on abstractions rather than concrete classes.

How It Works Internally

Unlike class inheritance, which uses a direct Method Table (vtable) offset lookup, interface calls use an Interface Table (itable).

- Each concrete class maintains an itable mapping interface method slots to concrete vtable slots.
- At runtime, calling a method via an interface pointer causes the .NET CLR to perform dynamic interface dispatch.
- The runtime resolves the target class's itable entry to locate the true class method address, enabling polymorphic behavior without common base class inheritance.

Key Rules & Terminologies

- Implicit Implementation: Interface methods are declared as public members on the target class and can be invoked directly from concrete instance variables.
- Explicit Implementation: Interface methods are prefixed with the interface name (e.g., void ILogger.Log()). They can only be invoked when the object is cast to the interface type, hiding API pollution on concrete

instances.

- Multiple Implementation: C# classes allow inheriting from only one base class, but implementing multiple interfaces.
- Default Interface Members (C# 8+): Interfaces can supply a default implementation for members, allowing API evolution without breaking existing implementations.

What Happens If Done Wrong

Bypassing interfaces leads to tightly coupled code bases where mock-based unit testing is impossible. Replacing infrastructure components (e.g., changing SQL Server storage to Redis) requires rewriting business logic across the entire application.

```
using System;

public interface INotifier
{
    void SendNotification(string message);
}

public class EmailNotifier : INotifier
{
    public void SendNotification(string message)
    {
        Console.WriteLine($"[Email Sent]: {message}");
    }
}

public class Program
{
    public static void Main()
    {
        INotifier notifier = new EmailNotifier();
        notifier.SendNotification("System maintenance scheduled at midnight.");
    }
}
```

? Real Project Example

In production ASP.NET Core applications, interfaces decouple web controllers from underlying infrastructure like payment processors, allowing seamless unit testing and runtime provider switching.

```
using Microsoft.AspNetCore.Mvc;
```

```
using Microsoft.Extensions.Logging;
using System;

namespace PaymentApi.Services;

public record PaymentRequest(decimal Amount, string Currency, string CustomerId);
public record PaymentResponse(bool IsSuccess, string TransactionId, string ErrorMessage);

public interface IPaymentGateway
{
    PaymentResponse ProcessPayment(PaymentRequest request);
}

public class StripePaymentGateway : IPaymentGateway
{
    private readonly ILogger<StripePaymentGateway> _logger;

    public StripePaymentGateway(ILogger<StripePaymentGateway> logger)
    {
        _logger = logger;
    }

    public PaymentResponse ProcessPayment(PaymentRequest request)
    {
        try
        {
            _logger.LogInformation("Processing Stripe payment for {Amount} {Currency}", request.Amount, request.Currency);
            // Simulate Stripe API Integration
            return new PaymentResponse(true, $"str_tx_{Guid.NewGuid():N}", string.Empty);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Stripe processing failed");
            return new PaymentResponse(false, string.Empty, ex.Message);
        }
    }
}

[ApiController]
[Route("api/[controller]")]
public class PaymentsController : ControllerBase
{
    private readonly IPaymentGateway _paymentGateway;

    public PaymentsController(IPaymentGateway paymentGateway)
    {
        _paymentGateway = paymentGateway;
    }

    [HttpPost]
    public IActionResult CreatePayment([FromBody] PaymentRequest request)
```

```
    {
        var result = _paymentGateway.ProcessPayment(request);
        if (!result.IsSuccess)
        {
            return BadRequest(new { Error = result.ErrorMessage });
        }
        return Ok(result);
    }
}
```

How It Works & Senior Guidelines

PaymentsController accepts IPaymentGateway in its constructor via ASP.NET Core's Dependency Injection framework.

The controller does not know or care whether StripePaymentGateway or a mock gateway handles the operation.

Senior Insight: Register this service in DI using builder.Services.AddScoped<IPaymentGateway, StripePaymentGateway>();. For unit tests, substitute IPaymentGateway with a mock object using libraries like Moq or NSubstitute to isolate controller tests from network dependencies.

⚠ Top 3 Mistakes

1. Fat Interfaces (Violating the Interface Segregation Principle)

Putting unrelated responsibilities into a single interface forces implementing classes to write stub/dummy implementations for unused methods.

❌ Bad Code:

```
public interface IDataStore
{
    void SaveData(string payload);
    string ReadData(int id);
    void SendEmailReceipt(string email); // Unrelated responsibility!
}
```

✅ Good Fix:

```
public interface IDataStore
{
    void SaveData(string payload);
```

```
    string ReadData(int id);
}

public interface IReceiptService
{
    void SendEmailReceipt(string email);
}
```

---

2. Leaking Implementation Details into Interface Definitions

Exposing persistence details like SqlDataReader or Entity Framework types in interfaces ties abstract callers directly to low-level infrastructure libraries.

 Bad Code:

```
using Microsoft.Data.SqlClient;

public interface IUserRepository
{
    SqlDataReader GetUserRaw(int id); // Tightly coupled to SQL Server!
}
```

 Good Fix:

```
public interface IUserRepository
{
    UserDto? GetUserById(int id); // Decoupled Domain/Data Transfer Object
}
```

---

3. Confusing Explicit Interface Implementation Access Modifiers

Explicitly implemented interface methods cannot have access modifiers (public/private) and cannot be called directly from an instance variable of the concrete type.

 Bad Code:

```
public interface ILogger
{
    void Log(string msg);
}
```

```
public class ConsoleLogger : ILogger
{
    public void ILogger.Log(string msg) // Compiler Error: Explicit implementation cannot specify modifiers
    {
        Console.WriteLine(msg);
    }
}
```

 Good Fix:

```
public class ConsoleLogger : ILogger
{
    void ILogger.Log(string msg) // Correct: No modifier, accessible when cast to ILogger
    {
        Console.WriteLine(msg);
    }
}
```

Industry News

- GitHub availability report: July 2026

GitHub published its operational status update detailing uptime, system incidents, and infrastructure resilience measures taken throughout the month. Tracking enterprise availability metrics helps software architects design fault-tolerant external integration patterns, such as implementing circuit breakers for third-party REST and interface contracts.

 Read full article

- Write your first prompt with the GitHub Copilot app

GitHub introduced hands-on guidance for engineering teams using the Copilot standalone app to write system prompts. Understanding AI interaction techniques enables developers to rapidly draft boilerplate interface implementations and generate standard mock dependencies during test suite construction.

 Read full article

- Your contributors are AI-first now. Is your project?

Open-source maintainers are structuring repositories around AI-driven workflows by standardizing interface contracts, strict typing, and comprehensive docs. Standardizing interface abstractions simplifies automated code generation because AI tools can reason accurately over explicit structural boundaries.

 Read full article

- Instructions Hygiene – What Frontier Models Still Need You to Say

Microsoft .NET team engineers explored boundary prompt optimizations and code generation clarity for LLMs. High instruction hygiene directly mirrors clean interface contract design in C#, where unambiguous input/output parameters minimize systemic logic bugs across distributed boundaries.

Read full article

- Netflix Adopts Cloud-Native Job Queueing System Kueue to Replace an In-House Solution

Netflix successfully migrated its massive batch scheduling pipelines away from bespoke in-house software to Kubernetes-native Kueue. Standardizing interface contracts allowed Netflix developers to swap foundational compute engine components cleanly without breaking higher-level internal orchestration jobs.

Read full article

- Spotify Builds External Index to Enable Low Latency Point Queries on Its Data Lake

Spotify designed custom indexing infrastructure sitting over massive object storage to speed up key-value data lookups. Loose coupling through standard storage API interfaces allowed Spotify engineers to attach an external indexing layer transparently without modifying underlying data ingestion pipelines.

Read full article

- Article: InfoQ Cloud and DevOps Trends Report - 2026

InfoQ released its yearly architecture radar analyzing cloud-native development practices, platform engineering, and serverless shifts. Software architects must rely heavily on interface-driven designs to prevent infrastructure lock-in when migrating microservices between competing cloud ecosystem providers.

Read full article

Interview Questions & Answers

Q1: What is an interface in C#, and how does it differ from an abstract class?

A1: An interface defines a pure contract containing no instance state, whereas an abstract class can contain state (fields), constructor logic, and full method implementations. A C# class can implement multiple interfaces, but it can inherit from only one base class.

```
public interface IDrawable { void Draw(); }
public abstract class Shape { public string Color { get; set; } = "Red"; }
```

Q2: What is explicit interface implementation, and when should you use it?

A2: Explicit interface implementation forces a member to be accessed only when the instance is explicitly cast

to the interface type. You use it to resolve member name collisions between two interfaces or to keep interface-specific methods hidden from the concrete object's primary public surface.

```
public interface IControl { void Paint(); }
public interface ISurface { void Paint(); }
public class Canvas : IControl, ISurface {
    void IControl.Paint() { }
    void ISurface.Paint() { }
}
```

Q3: Can an interface contain method implementations in modern C#?

A3: Yes, starting with C# 8.0, interfaces support Default Interface Members (DIMs). This allows developers to add default body implementations to interfaces without breaking existing concrete classes that implement those interfaces.

```
public interface ILogger {
    void Log(string message) => Console.WriteLine($"Default: {message}");
}
```

Q4: How do interfaces enable runtime polymorphism and Dependency Injection in .NET?

A4: Interfaces decouple client code from concrete class types by acting as a abstraction layer. The ASP.NET Core Dependency Injection container maps an interface contract to a concrete implementation at startup, resolving dependencies automatically at runtime and making components easily swappable during testing.

```
builder.Services.AddScoped<IService, ConcreteService>();
```

Q5: How does the .NET CLR resolve interface method calls internally?

A5: The CLR resolves interface calls using an Interface Table (itable). Each concrete type has an itable mapping interface slots to its virtual method table (vtable). When calling a method via an interface variable, the CLR uses dynamic interface dispatch to look up the itable slot for that instance at runtime.

Q6: What is the Interface Segregation Principle (ISP), and how do you update a widely used interface without breaking callers?

A6: ISP states that clients should not be forced to depend on methods they do not use. To update an existing interface safely without breaking downstream implementations, you should create a secondary extension interface (e.g., IServiceV2) or provide Default Interface Members (DIMs) in C# 8+.

```
public interface IProcessor { void Process(); }
public interface IProcessorV2 : IProcessor { void ProcessAsync(); }
```

? Revision Summary

Day 6: Inheritance

Inheritance allows derived classes to inherit state and behavior from a single base class using virtual and override semantics.

- Key Idea: Models "is-a" relationships to share implementation logic across hierarchical domain entities.
- One Thing to Remember: Always prefer composition and interface implementation over deep class inheritance hierarchies to prevent rigid, tightly coupled models.

Day 4: Methods

Methods encapsulate executable statements into reusable blocks, identified uniquely by their signature (name, parameter types, and parameter modifiers).

- Key Idea: Provides modular abstractions with parameter modifiers like ref, out, and in for precise memory handling.
- One Thing to Remember: Method overload resolution is determined entirely by parameter counts and types, never by the return type alone.

? Tomorrow Preview

Tomorrow we explore Abstract Classes & Abstract Methods. You will learn how to combine common reusable base state with enforced abstract method contracts to construct the template method pattern in C#.